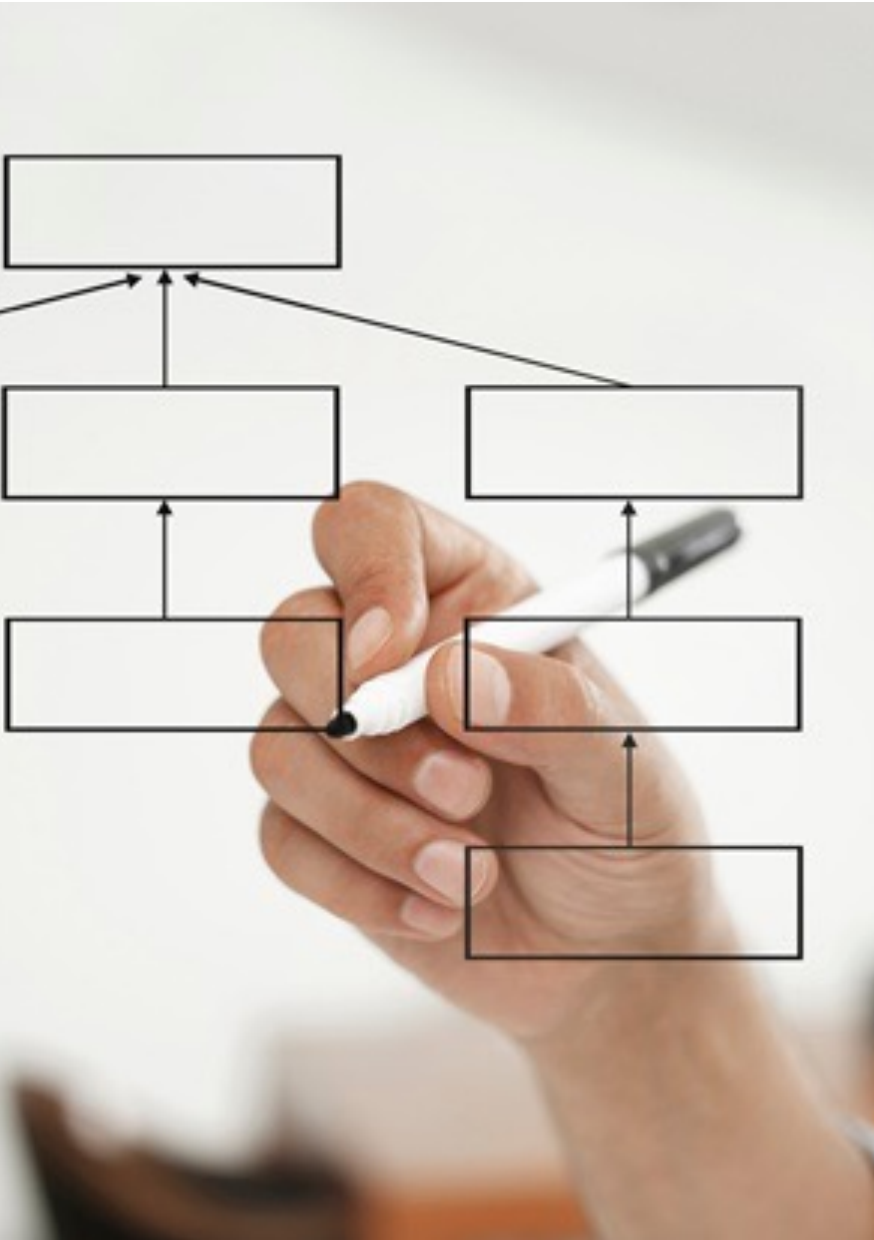




Estrutura de Dados I

CÁSSIO CAPUCHO PEÇANHA - 14

Problema

- Princípio de funcionamento dos métodos de busca
 - Procurar a informação desejada com base na comparação de suas chaves, isto é com base em algum valor que a compõe
- Problema
 - Algoritmos eficientes necessitam que os elementos estejam armazenados de forma ordenada
 - Custo ordenação melhor caso é $O(N \log N)$
 - Custo da busca melhor caso é $O(\log N)$

Problema

- Custo da comparação de chaves é alto
- O que seria uma operação de **busca ideal**?
 - Seria aquela que permitisse o acesso direto ao elemento procurado, sem nenhuma etapa de comparação de chaves
 - Nesse caso, teríamos um custo $O(1)$
 - Tempo sempre constante de acesso

Problema

- Uma saída é usar **arrays**

- São estruturas que utilizam índices para armazenar informações
- Permite acessar uma determinada posição com custo $O(1)$

- **Problema**

- Arrays não possuem nenhum mecanismo que permita calcular a posição onde uma informação está armazenada
- A operação de busca não é $O(1)$

Problema

Precisamos do tempo de acesso do array juntamente com a capacidade de busca um elemento em tempo constante

Solução: usar uma **tabela hash**

Tabela Hash

- Também conhecidas como tabelas de indexação ou de espalhamento
 - É uma generalização da idéia de array.
- Idéia central
 - Utilizar uma função, chamada de **função de hashing**, para espalhar os elementos que queremos armazenar na tabela.
 - Esse espalhamento faz com que os elementos fiquem dispersos de forma não ordenada dentro do array que define a tabela.
- Exemplo: Imagine que a matrícula de um aluno é a chave. A função de hashing "traduz" essa matrícula para uma posição na tabela, permitindo encontrar a nota do aluno de forma instantânea.

A Função de Hashing

➤ O que é?

- Uma função que distribui os dados de forma equilibrada pela tabela.

➤ Características de uma boa função:

- Fácil de calcular.
- Eficaz em gerar posições diferentes para chaves diferentes.

➤ Exemplo: Método da Divisão

- A função é $h(\text{chave}) = \text{chave} \pmod{\text{tamanho_da_tabela}}$

➤ Exemplo:

- Em uma tabela de 10 posições, a chave 255 é mapeada para a posição $255 \pmod{10} = 5$

Tabela Hash

Exemplo:

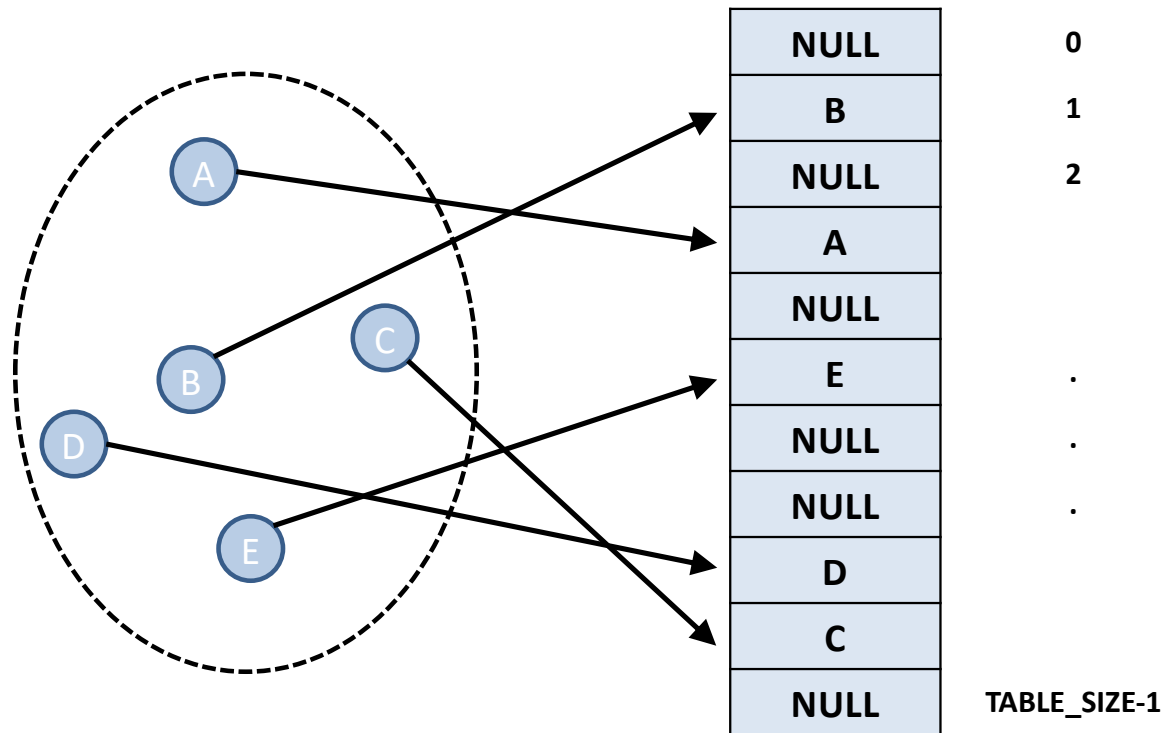


Tabela Hash

- Por que espalhar os elementos melhora a busca?
 - A tabela permite associar valores a chaves
 - **chave**: parte da informação que compõe o elemento a ser inserido ou buscado na tabela
 - **valor**: é a posição (índice) onde o elemento se encontra no array que define a tabela
 - Assim, a partir de uma **chave** podemos acessar de forma rápida uma determinada **posição** do array
 - Na média, essa operação tem custo $O(1)$

Tabela Hash

➤ Vantagens

- Alta eficiência na operação de busca
 - Caso médio é $O(1)$ enquanto o da busca linear é $O(N)$ e a da busca binária é $O(\log_2 N)$
- Tempo de busca é praticamente independente do número de chaves armazenadas na tabela
- Implementação simples

Tabela Hash

- Infelizmente, esse tipo de implementação também tem suas desvantagens
 - Alto custo para recuperar os elementos da tabela ordenados pela chave.
 - Nesse caso, é preciso ordenar a tabela
 - O pior caso é $O(N)$, sendo N o tamanho da tabela
 - Alto número de colisões

Tabela Hash - Colisão

➤ O que é uma **colisão**?

- Uma colisão ocorre quando duas (ou mais) chaves diferentes tentam ocupar a mesma posição na tabela hash.
- A colisão de chaves não é algo exatamente ruim, é apenas algo indesejável pois diminui o desempenho do sistema.

➤ Exemplo:

- Em uma tabela de 10 posições, a chave 255 e a chave 355 geram a mesma posição:

➤ $255 \pmod{10} = 5$

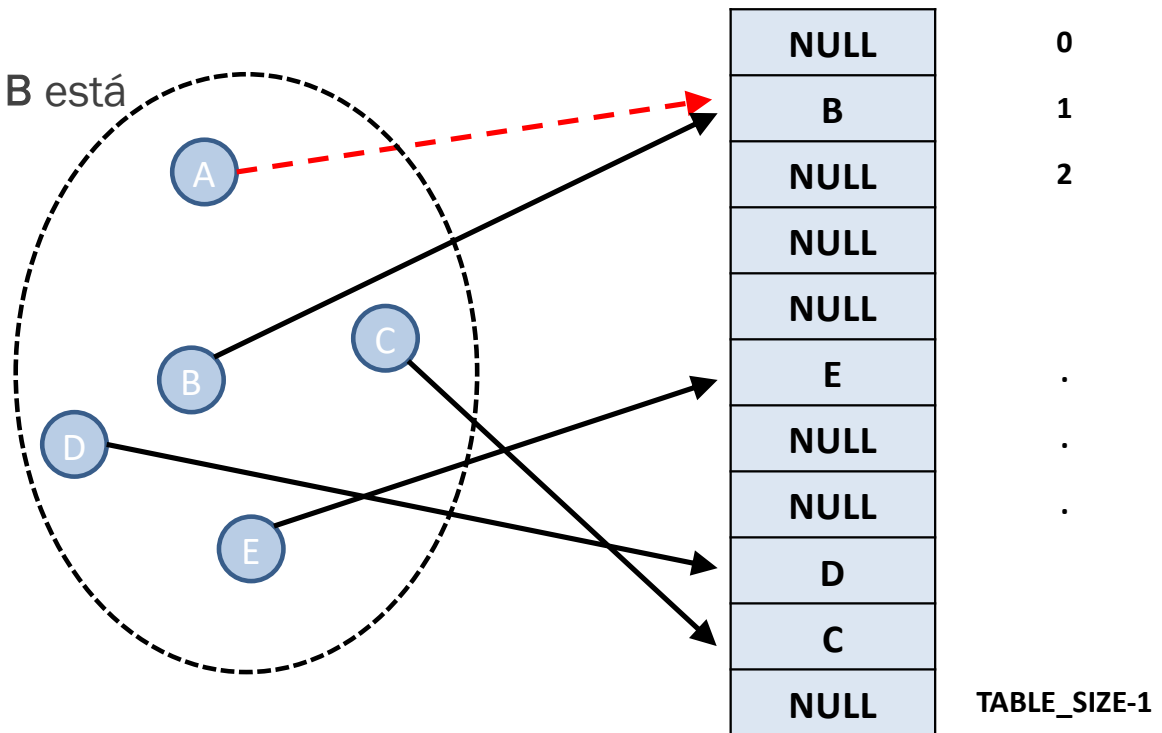
➤ $355 \pmod{10} = 5$

- Consequência: As colisões diminuem a eficiência da tabela, e por isso precisam ser resolvidas.

Tabela Hash

Exemplo de **colisão**

- A tenta ocupar a posição onde B está



Função de Hashing

- Inserção e busca: é necessário calcular a posição dos dados dentro da tabela.
- Função de Hashing
 - Calcula a posição a partir de uma chave escolhida a partir dos dados manipulados



Função de Hashing

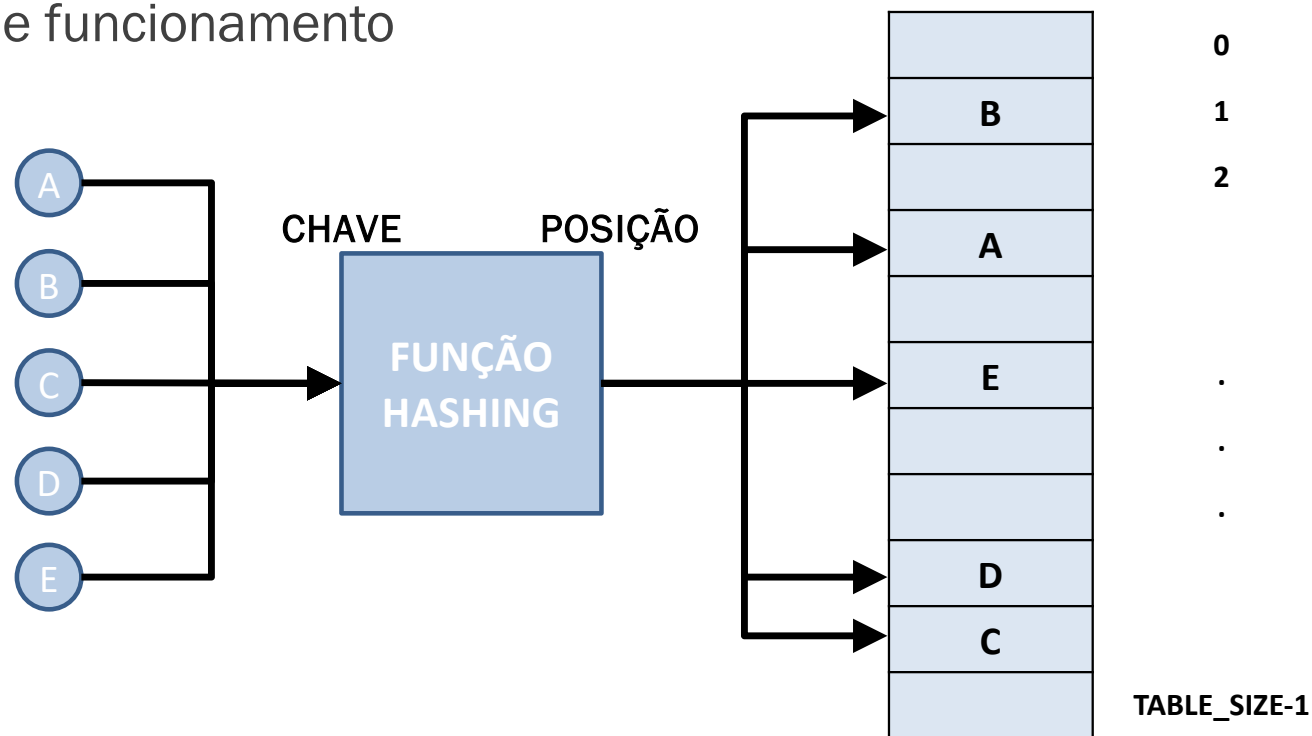
➤ Função de Hashing

- É extremamente importante para o bom desempenho da tabela.
- Ela é responsável por distribuir as informações de forma equilibrada pela tabela hash



Função de Hashing

Exemplo de funcionamento



Função de Hashing

- Para um bom funcionamento, deve satisfazer as seguintes condições
 - Ser simples e barata de se calcular
 - Garantir que valores diferentes produzam posições diferentes
 - Gerar uma distribuição equilibrada dos dados na tabela
 - Cada posição da tabela tem a mesma chance de receber uma chave (máximo espalhamento)

Função de Hashing

- Sua implementação depende do conhecimento prévio da natureza e domínio da chave a ser utilizada
 - Exemplo: utilizar apenas três dígitos do número de telefone de uma pessoa para armazená-lo na tabela.
 - Neste caso, seria melhor usar os três últimos dígitos do que os três primeiros, pois os primeiros costumam se repetir com maior frequência e iriam gerar posições iguais na tabela.
 - Assim, o ideal é usar um cálculo diferente de Hash para cada tipo de chave.

Função de Hashing

- Alguns exemplos de função de hashing comumente utilizadas
 - Método da Divisão
 - Método da Multiplicação
 - Método da Dobra

Função de Hashing

➤ Método da Divisão

- Ou método da congruência linear

- Consiste em calcular o **resto da divisão** do valor inteiro que representa o elemento pelo tamanho da tabela, **TABLE_SIZE**

- Simples e direta

- $h(chave) = chave \bmod tamanho_tabela$

- Simples, mas pode gerar colisões se o tamanho da tabela for mal escolhido (evite potências de 2; prefira primos).

Função de Hashing

➤ Método da Divisão

- Apesar de simples, apresenta alguns problemas.
- Resto da divisão: valores diferentes podem resultar na mesma posição
- Exemplo
 - O resto da divisão de 11 por 10 e de 21 por 10 são o mesmo valor de posição: 1
 - Uma maneira de reduzir esse tipo de problema é utilizar como tamanho da tabela, `TABLE_SIZE`, um número primo.

Função de Hashing

➤ Método da Multiplicação

- Também chamado de método da congruência linear multiplicativo
- Usa uma constante fracionária A , $0 < A < 1$, para multiplicar o valor da chave que representa o elemento.
- Em seguida, a parte fracionária resultante é multiplicada pelo tamanho da tabela para calcular a posição do elemento

➤ $h(chave) = \lfloor m * (chave * A \bmod 1) \rfloor$

- m = tamanho da tabela
- A = número real ($0 < A < 1$), ex: $A = (\sqrt{5} - 1)/2 \approx 0,618$
- Mais difícil de implementar, mas distribui melhor.

Função de Hashing

➤ Método da Multiplicação

- Exemplo: calcular a posição da chave 123456, usando a constante fracionária $A = 0,618$ e que o tamanho da tabela seja 1024

```
posição = ParteInteira(TABLE_SIZE * ParteFracionária(chave * A))
```

```
posição = ParteInteira(1024 * ParteFracionária(123456 * 0,618))
```

```
posição = ParteInteira(1024 * ParteFracionária(762950,808))
```

```
posição = ParteInteira(1024 * 0,808)
```

```
posição = ParteInteira(827,392)
```

```
posição = 827
```

Função de Hashing

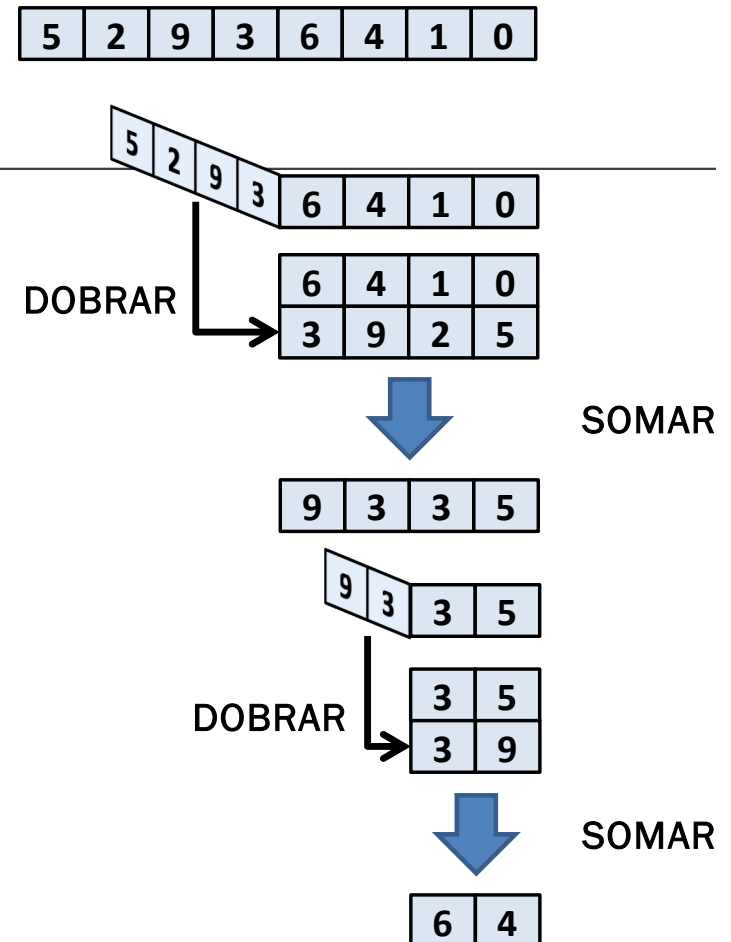
➤ Método da Dobra

- Utiliza um esquema de dobrar e somar os dígitos do valor para calcular a sua posição.
- Considera o valor inteiro que representa o elemento como uma sequência de dígitos escritos num pedaço de papel.
- Enquanto esse valor for maior que o tamanho da tabela, o papel é dobrado e os dígitos sobrepostos são somados, desconsiderando-se as dezenas
- Note que este processo deve ser repetido enquanto os dígitos formarem um número maior que o tamanho da tabela.

Função de Hashing

➤ Método da Dobra

➤ Exemplo



Hashing Universal

- **Hashing universal** é um método para escolher aleatoriamente uma função hash a partir de uma família de funções, de forma que a chance de colisão entre duas chaves seja pequena.
- Em muitos casos, se alguém souber qual é a função hash usada, pode deliberadamente causar colisões (ataques ou pior desempenho).
 - Em vez de usar uma função hash fixa, ele escolhe aleatoriamente uma função de uma família de funções.
 - Isso torna muito improvável que duas chaves colidam.
- É usado em contextos onde segurança e desempenho garantido são importantes.

Hashing imperfeito e perfeito

➤ Hashing Perfeito:

- Ocorre quando nunca há colisões. Chaves diferentes sempre produzem posições diferentes.
- As operações de busca e inserção têm custo $O(1)$ no pior caso.
- É ideal para aplicações onde colisões não podem ser toleradas, como em palavras reservadas de uma linguagem de programação.

➤ Hashing Imperfeito:

- Ocorre quando a mesma posição é gerada para duas chaves diferentes, resultando em colisões.
- É o que acontece na maioria dos casos práticos.

Tratamento de Colisões

- Colisões são teoricamente inevitáveis. Por isso, devemos sempre ter uma abordagem para tratá-las.
 - Existem diversas formas de se tratar a colisão
 - Duas técnicas muito comuns
 - endereçamento aberto
 - encadeamento separado

Endereçamento Aberto

➤ Definição

- Também conhecido como *open addressing* ou *rehash*
- No caso de um **colisão**, percorrer a tabela hash buscando por uma **posição** ainda não ocupada
- Os elementos são armazenados na própria tabela hash
 - Evita o uso de listas encadeadas

Endereçamento Aberto

➤ A tenta ocupar a posição de B

➤ Devemos percorrer a tabela até achar uma posição vaga (NULL)

A

NULL	0
B	1
NULL	2
NULL	
NULL	
E	.
NULL	.
NULL	.
D	
C	
NULL	TABLE_SIZE-1

Endereçamento Aberto

➤ Vantagens

- Maior número de posições na tabela para a mesma quantidade de memória usada no **encadeamento separado**
- A memória utilizada para armazenar os ponteiros da lista encadeada no **encadeamento separado** pode ser aqui usada para aumentar o tamanho da tabela, diminuindo o número de colisões

Endereçamento Aberto

Vantagens

- Busca é realizada dentro da própria tabela
 - Recuperação mais rápida de elementos
- Voltada para aplicações com restrições de memória
- Ao invés de acessarmos ponteiros extras, calculamos a sequência de posições a serem armazenadas.

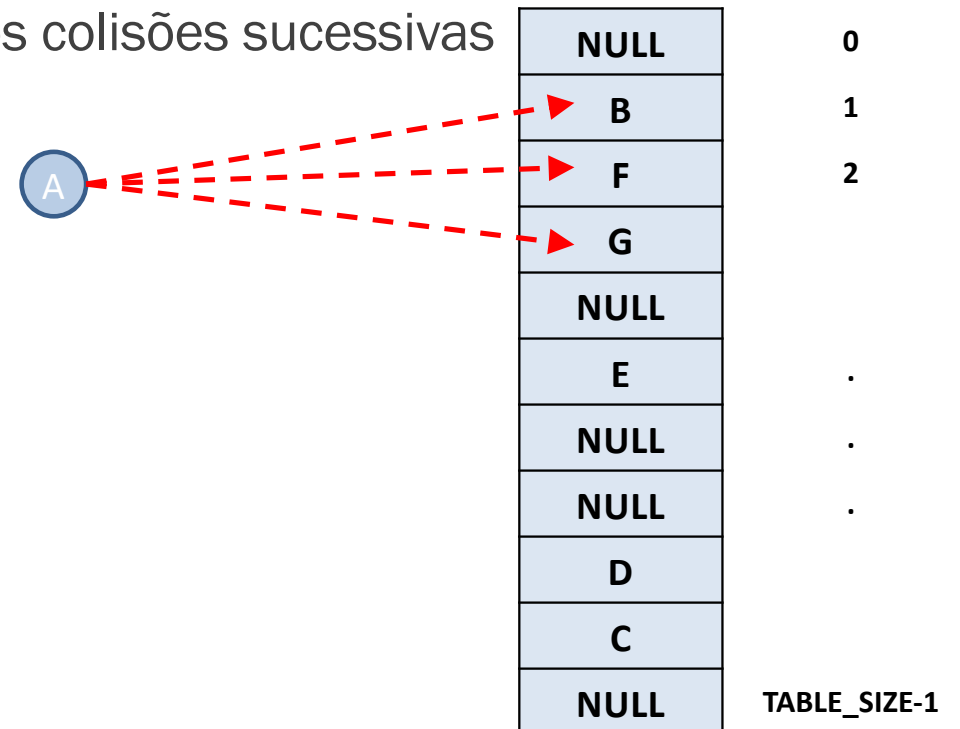
Endereçamento Aberto

Desvantagens

- Maior esforço de processamento no cálculo das posições
- Esse esforço maior se deve ao fato de que, quando uma colisão ocorre, devemos calcular uma nova posição da tabela
- Colisões sucessivas

Endereçamento Aberto

Se apenas percorrermos o array, teremos colisões sucessivas



Endereçamento Aberto

- Para a realização do cálculo da nova posição após a colisão, existem três estratégias muito utilizadas
 - **Sondagem Linear**: Busca a próxima posição de forma sequencial (ex: Posição + 1, Posição + 2, ...). Pode gerar "agrupamentos", prejudicando o desempenho.
 - **Sondagem Quadrática**: Busca a próxima posição de forma não linear (ex: Posição + 1^2 , Posição + 2^2 , ...). Ajuda a evitar agrupamentos.
 - **Duplo Hash**: Usa uma segunda função de hashing para determinar o "salto" em caso de colisão. É a mais eficaz para distribuir os dados.

Endereçamento Aberto

Sondagem linear

(ex: Posição + 1, Posição + 2, ...)

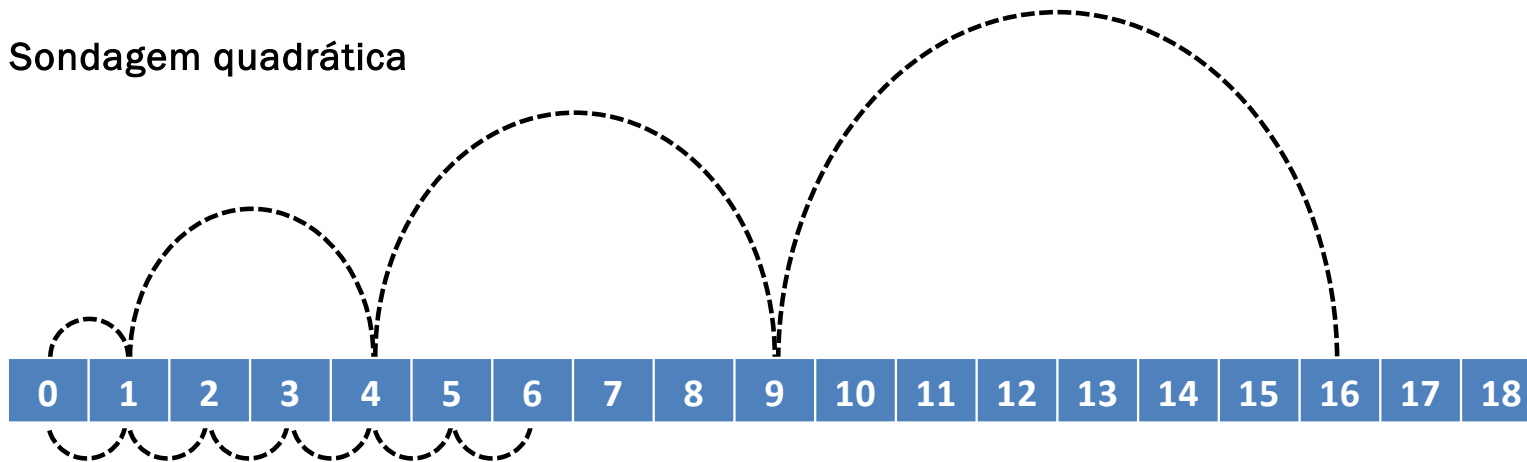
NULL	0	CHAVE	POSIÇÃO	INSERÇÃO	E	0
NULL	1	A	2	Posição 2 vazia. Inere elemento	NULL	1
NULL	2	B	6	Posição 6 vazia. Inere elemento	A	2
NULL	3	C	2	Posição 2 ocupada, procura na próxima posição: 3	C	3
NULL	4			Posição 3 vazia. Inere elemento	NULL	4
NULL	5	D	10	Posição 10 vazia. Inere elemento	NULL	5
NULL	6	E	10	Posição 10 ocupada, procura na próxima posição. Como a posição 10 é a última, volta para o início: 0	B	6
NULL	7				NULL	7
NULL	8				NULL	8
NULL	9				NULL	9
NULL	10				D	10



Endereçamento Aberto

Sondagem Quadrática

Sondagem quadrática



Sondagem linear

ex: Posição + 1^2 , Posição + 2^2 , ...

Endereçamento Aberto

Duplo Hash

- Tenta espalhar os elementos utilizando duas funções de hashing:
 - a primeira função de hashing, $H1$, é utilizada para calcular a posição inicial do elemento
 - a segunda função de hashing, $H2$, é utilizada para calcular os deslocamentos em relação a posição inicial (no caso de uma colisão)
- A posição de um novo elemento na tabela hash é obtida como sendo
 - $H1 + i * H2$
 - onde i é tentativa atual de inserção do elemento
- É necessário que as duas funções de hashing sejam diferentes.
 - A segunda função de hashing não pode resultar em um valor igual a ZERO pois, neste caso, não haveria deslocamento
- Funcionamento
 - Primeiro elemento ($i = 0$) é colocado na posição obtida por $H1$
 - Segundo elemento (colisão) é colocado na posição $H1 + 1 * H2$
 - Terceiro elemento (nova colisão) é colocado na posição $H1 + 2 * H2$

Tratamento de Colisões - Endereçamento Aberto

- Conceito: Se a posição calculada estiver ocupada, o algoritmo busca a próxima posição vaga na própria tabela.
- Vantagem: Não usa estruturas de dados adicionais, economizando memória.
- Estratégias para encontrar a próxima posição:
 - Sondagem Linear: Busca a próxima posição de forma sequencial (ex: Posição + 1, Posição + 2, ...). Pode gerar "agrupamentos", prejudicando o desempenho.
 - Sondagem Quadrática: Busca a próxima posição de forma não linear (ex: Posição + 1^2 , Posição + 2^2 , ...). Ajuda a evitar agrupamentos.
 - Duplo Hash: Usa uma segunda função de hashing para determinar o "salto" em caso de colisão. É a mais eficaz para distribuir os dados.

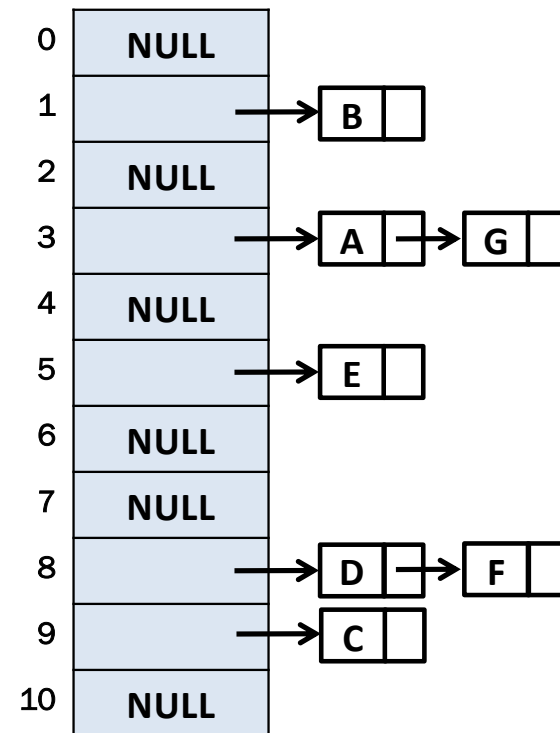
Encadeamento Separado

- Também conhecido como *separate chaining*
 - Não procura por posições vagas (valor **NULL**) dentro do array que define a tabela
 - Armazena dentro de cada posição do array o início de uma lista dinâmica encadeada
 - É dentro dessa lista que serão armazenadas as colisões (elementos com chaves iguais) para aquela posição do array

Encadeamento Separado

Exemplo

CHAVE	POSIÇÃO
A	3
B	1
C	9
D	8
E	5
F	8
G	3



Encadeamento Separado

➤ Características

- A lista dinâmica encadeada mantida em cada posição da tabela pode ser ordenada ou não
- Lista não ordenada
 - Inserção tem complexidade $O(1)$ no pior caso: basta inserir o elemento no início da lista.
 - Busca tem complexidade $O(M)$ no pior caso: busca linear

➤ Desvantagem

- Quantidade de memória consumida: gastamos mais memória para manter os ponteiros que ligam os diferentes elementos dentro de cada lista

Tratamento de Colisões - Encadeamento Separado

- Conceito: Cada posição da tabela armazena o início de uma lista encadeada. Os elementos que colidem são adicionados a essa lista.
- Vantagem: A inserção é rápida, com custo $O(1)$.
- Desvantagem: O consumo de memória é maior devido aos ponteiros das listas.
- Exemplo: A posição 5 da tabela teria uma lista encadeada com a chave 255 e a chave 355.